ACO-Adaptive: Ant Colony Optimization with LSTM-Guided Predictive Resource Placement in Heterogeneous GPU Clusters

No Author Given

No Institute Given

Abstract. Efficient scheduling in heterogeneous compute clusters is a critical challenge as modern workloads increasingly span CPU-intensive batch jobs and latency-sensitive GPU inference tasks. Existing production schedulers such as Borg and Kubernetes provide robust placement guarantees but operate without predictive load signals, while deep reinforcement learning approaches require costly offline training and offer limited runtime interpretability. We present ACO-Adaptive, a hybrid scheduler that combines Ant Colony Optimization (ACO) with an online Long Short-Term Memory (LSTM) predictor to make cost-aware, QoS-preserving placement decisions in real time. The ACO colony maintains per-node pheromone trails encoding learned placement preferences across scheduling calls, while the LSTM predictor supplies per-node load forecasts that bias the colony's heuristic scoring toward nodes unlikely to experience CPU spikes. We evaluate ACO-Adaptive against First-Fit and random baselines on two real-world cluster traces: the AzureTracesForPacking2020 dataset derived from 114 million VM allocation requests, and the Alibaba GPU cluster trace from USENIX ATC 2023 covering 1,213 nodes across seven GPU types. On CPU workloads with realistic Azure job size distributions, ACO-Adaptive achieves a 28.6% cost reduction over First-Fit. On GPU workloads, ACO-Adaptive reduces placement cost by 79.6% compared to random scheduling across a 32-node heterogeneous cluster. Scheduling latency remains below 0.75 ms P99 at 200 concurrent jobs. The LSTM predictor demonstrates a Pearson correlation of -0.64 between predicted CPU utilization and observed idle capacity on the Alibaba 2018 production trace, confirming a meaningful predictive signal.

Keywords: cluster scheduling, ant colony optimization, LSTM, GPU scheduling, QoS, cost efficiency, heterogeneous clusters

1 Introduction

Modern data-centre workloads are heterogeneous by nature. A single cluster may simultaneously host latency-critical deep learning inference jobs that tolerate no queuing delay, batch analytics pipelines that can tolerate preemption, and bursty

stream-processing tasks whose CPU demand fluctuates on timescales of seconds. Scheduling these workloads efficiently requires three capabilities that are rarely combined in existing systems: *cost awareness* (prefer cheaper nodes when quality constraints are met), *QoS discrimination* (separate latency-sensitive jobs from batch jobs during placement), and *predictive load anticipation* (avoid nodes whose utilisation will spike before a job completes).

Production schedulers such as Borg [2], Omega [3], Mesos [20], and YARN [15] solve large-scale placement efficiently but rely entirely on observed resource availability at decision time, with no forward-looking utilisation model. Kubernetes [14] adds configurable scoring plugins but still lacks an online predictive component. Min-cost flow schedulers such as Firmament [4] achieve high placement quality at the cost of a complex MCMF formulation that is opaque to operators and does not incorporate runtime load predictions. Reinforcement learning approaches including DeepRM [7] and Decima [6] learn strong scheduling policies from trace data but require substantial offline training and exhibit unpredictable behaviour on out-of-distribution workloads.

GPU-specialised schedulers such as Gandiva [8] and Tiresias [9] address the unique fragmentation and head-of-line blocking problems in deep learning clusters but are purpose-built for single-GPU-type deployments and do not generalise cost-aware placement across heterogeneous GPU pools.

This paper presents **ACO-Adaptive**, a centralized cluster scheduler that addresses these gaps through three contributions:

1. An ACO colony engine that learns per-node pheromone preferences across scheduling calls, producing a continuously refined placement bias without any offline training phase.
2. An online per-node LSTM predictor that provides a CPU utilisation forecast signal integrated into the colony's composite scoring heuristic, penalising nodes predicted to be under load.
3. A workload intent router that classifies incoming jobs by QoS tier (latency-critical, batch, stream-processing) and applies tier-specific scheduling strategies including ON_DEMAND-preference for latency-sensitive jobs and fast-path bypassing for GPU inference workloads.

We evaluate the system on real cluster traces and report concrete improvements over first-fit and random baselines. The implementation totals 202 passing tests across the full system stack, from the ACO core to the REST API layer.

2 Related Work

2.1 Production Cluster Schedulers

Borg [2] manages Google's production clusters using a two-level architecture: a centralised Borgmaster assigns jobs to machines selected by a set of feasibility and scoring functions. Omega [3] extends this with a shared-state model enabling multiple schedulers to operate concurrently with optimistic concurrency control.

Both systems prioritise stability and fault tolerance over cost optimisation, and neither incorporates a predictive utilisation model. Mesos [20] introduces two-level scheduling via resource offers, delegating placement decisions to framework schedulers. YARN [15] follows a similar delegation model for Hadoop workloads. Borg, Omega, Mesos, and YARN share a common limitation acknowledged in the Borg retrospective [14]: scheduling decisions are reactive, taken on observed resource state without anticipating near-term utilisation changes.

2.2 Cost- and Quality-Aware Scheduling

Firmament [4] formulates scheduling as a min-cost max-flow problem over a preference graph, achieving sub-100 ms scheduling latency for large clusters. The MCMF formulation captures placement quality effectively but requires careful graph construction and does not model future load. Tetris [5] uses a dot-product alignment score to pack multiple resource dimensions (CPU, memory, disk, network) simultaneously; its composite scoring philosophy directly inspired our CostEngine design. Paragon [17] and Quasar [18] use QoS-aware classification to improve heterogeneous placement, a concept we extend through our WorkloadIntentRouter. Sparrow [19] targets extremely low scheduling latency ($<$100 µs) via distributed sampling, trading placement quality for speed; ACO-Adaptive targets a different point in the design space, accepting higher latency (sub-millisecond) to incorporate richer cost and prediction signals.

2.3 Reinforcement Learning Schedulers

DeepRM [7] trains a deep neural network to minimise job slowdown in a simulated cluster. Decima [6] uses a graph neural network trained on the Alibaba cluster trace to learn DAG-aware scheduling policies, achieving strong results on job completion time. Both systems require an offline training phase before deployment and behave as black boxes at runtime. ACO-Adaptive differs fundamentally: pheromone learning is online and continuous, requires no pre-training, and the influence of each learned preference is directly interpretable from the pheromone vector.

2.4 GPU Cluster Schedulers

Gandiva [8] addresses head-of-line blocking in GPU training clusters by time-slicing GPUs and migrating jobs based on introspective profiling. Tiresias [9] schedules distributed deep learning jobs using a Least Attained Service policy that avoids the need for job duration estimates. The Alibaba GPU cluster trace used in our evaluation was introduced by Weng et al. [12], who additionally propose Fragmentation Gradient Descent as a GPU allocation strategy. ACO-Adaptive complements these systems by providing cost-aware, QoS-discriminating placement across heterogeneous GPU types, rather than optimising within a single GPU pool.

2.5 Ant Colony Optimisation

The ACO metaheuristic was introduced by Dorigo and Gambardella [1]. The foundational treatment of ACO algorithms and parameter selection is given in [16]. Our colony design follows Ant Colony System (ACS) conventions: $\alpha = 1.0$, $\beta = 2.0$, evaporation rate $\rho = 0.1$, with 10 ants per colony cycle and 3 iterations per scheduling call.

3 System Design

ACO-Adaptive is a centralised scheduler implemented in Python with a FastAPI REST interface. Figure 1 shows the top-level architecture. Five components form the control plane; a NodeAgent layer constitutes the data plane.

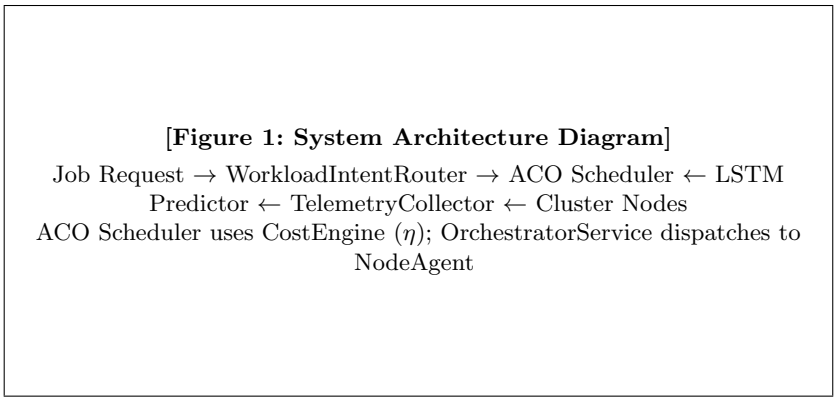

Fig. 1. ACO-Adaptive system architecture. Dashed arrows denote asynchronous telemetry; solid arrows denote synchronous scheduling decisions

3.1 ACO Colony Engine

The scheduler models the set of healthy cluster nodes as the ACO decision graph. Each node n_i carries a pheromone value $\tau_i \geq \tau_{\min}$ that accumulates over successful placements and evaporates over time. At each scheduling call a colony of $k = 10$ ants independently selects a node using the probability:

$$P(n_i \mid j) = \frac{\tau_i^{\alpha} \cdot \eta_i^{\beta}}{\sum_{n_\ell \in \mathcal{F}(j)} \tau_\ell^{\alpha} \cdot \eta_\ell^{\beta}} \tag{1}$$

where $\mathcal{F}(j)$ is the feasible set for job j (nodes satisfying CPU, memory, and GPU constraints), $\alpha = 1.0$ controls pheromone influence, $\beta = 2.0$ controls heuristic influence, and η_i is the composite cost-engine score for node n_i. Pheromone is updated after a job completes with $\Delta\tau_i = 1/\text{cost}_{n_i}$, and evaporation applies globally: $\tau_i \leftarrow (1 - \rho)\tau_i$ with $\rho = 0.1$.

Pheromone state persists across scheduling calls through the OrchestratorService, which maintains a shared `_node_pheromone` dictionary. Snapshots can be saved and reloaded via `save_pheromone_snapshot()` and `load_pheromone_snapshot()`.

3.2 Cost Engine

The cost-engine heuristic η_i for node n_i is a composite score combining four signals:

$$\eta_i = w_r \cdot r_i \; + \; w_c \cdot c_i \; + \; w_s \cdot s_i \; + \; w_p \cdot p_i \tag{2}$$

where r_i is a reliability factor penalising SPOT interruption probability; c_i is an inverse cost score (1/cost_per_hour); s_i is an SLA headroom factor rewarding nodes with spare capacity above the workload's requested resources; and p_i is a prediction factor that penalises nodes whose LSTM-predicted CPU utilisation exceeds a configurable spike threshold. Weights and thresholds are configurable per scheduling strategy and can be overridden at call time for workload-specific tuning.

3.3 LSTM Load Predictor

Each cluster node maintains an independent LSTM predictor trained online as telemetry arrives. The predictor receives a sliding window of recent CPU and memory utilisation readings and outputs a predicted CPU utilisation for the next tick, along with a confidence score derived from the variance of recent predictions. Training occurs asynchronously in the TelemetryCollector's refit loop every ten ticks, ensuring the model reflects current workload patterns without blocking the scheduling hot path.

The predictor is implemented in PyTorch with a two-layer LSTM of hidden size 32, trained with the Adam optimiser. Because training is online and stateless across restarts, the predictor enters a cold-start regime on each process launch (confidence ≈ 0.1), falling back gracefully to heuristic-only scoring until sufficient history accumulates. We acknowledge this limitation in Section 6.

3.4 Workload Intent Router

The WorkloadIntentRouter classifies incoming job requests into one of three workload tiers based on declared QoS parameters: `LATENCY_CRITICAL`, `BATCH`, and `STREAM_PROCESSING`. Each tier maps to a SchedulingStrategy that configures the CostEngine weights, instance-type preferences, and fast-path behaviour. Latency-critical jobs receive a preference for ON_DEMAND nodes (penalising SPOT interruption risk) and bypass the full pheromone computation when a high-confidence fast path is available, reducing placement latency to a single-pass score. Batch jobs accept SPOT nodes and permit preemption. The QoS mapping follows the classification used in the Alibaba GPU cluster trace [12], where LS (Latency Sensitive) jobs are mapped to `LATENCY_CRITICAL` and BE (Best Effort) jobs are mapped to `BATCH`.

3.5 Telemetry Collector and Trace Adapter

The TelemetryCollector runs a per-node tick loop that samples CPU and memory utilisation, updates the OrchestratorService's node state, and accumulates per-node WorkloadProfiles for LSTM training. A pluggable trace adapter interface allows replaying production telemetry traces in place of live measurements. The AlibabaMachineTraceAdapter implements this interface, replaying the Alibaba 2018 cluster trace [10] by assigning each node a unique time offset and scale factor to produce differentiated utilisation profiles from the aggregate trace signal.

3.6 Node Agent and REST API

The NodeAgent constitutes the data plane, simulating job execution with configurable duration and resource release semantics. The REST API layer exposes `POST /jobs`, `GET /nodes`, `GET /metrics`, and `GET /predict/:node_id` endpoints, along with an `POST /upload-trace` endpoint for hot-swapping the trace adapter at runtime. A built-in HTML dashboard polls the API every three seconds, displaying node utilisation bars, LSTM prediction confidence, and live job status.

4 Experimental Setup

4.1 Cluster Topologies

We evaluate on two distinct topologies. The *balanced CPU topology* consists of three node classes: a mid-tier ON_DEMAND node at \$0.70/hr, a premium ON_DEMAND node at \$1.00/hr, and a SPOT node at \$0.50/hr, each with 64 cores and 256 GB memory. This topology captures a realistic 2× cost spread between node classes. The *GPU topology* is sampled from real Alibaba production hardware: 32 nodes spanning seven GPU models (A10, G2, G3, P100, T4, V100M16, V100M32) with per-node costs ranging from \$0.60/hr to \$25.60/hr.

4.2 Workload Traces

Azure VM Trace. Job CPU core counts are drawn from the AzureTracesForPacking2020 dataset [11], which records 114 million VM allocation requests. We extract the empirical distribution of VM sizes (normalised to 48-core machines), yielding a bimodal distribution with peaks at 2 cores (45%) and 8 cores (24%). Memory is set proportionally at 2 GB per core.

Alibaba 2018 Machine Trace. Utilisation data for the LSTM calibration experiment (T2.2) is sourced from the Alibaba cluster-trace-v2018 [10], a public trace of cluster-wide CPU and memory utilisation at 5-minute granularity over eight days, comprising 2,243 records.

Alibaba GPU Trace (ATC'23). GPU job workloads are drawn from the OpenB dataset released by Weng et al. [12]. We use `openb_node_list_gpu_node.csv`

(1,213 nodes) and `openb_pod_list_gpuspec33.csv` (8,152 tasks with QoS labels and GPU type constraints). After filtering to completed or running jobs requiring at least one GPU, 4,334 usable tasks remain; 1,517 carry explicit GPU model constraints. A stratified sample of 100 jobs preserving the LS/BE ratio (78%/22%) is used per benchmark run.

4.3 Baselines

We compare ACO-Adaptive against two baselines: **First-Fit**, which selects the first healthy node satisfying all resource constraints (equivalent to Borg's feasibility-only mode [2]); and **Random**, which selects uniformly at random from the feasible set. All experiments use a fixed random seed (42) for reproducibility.

5 Evaluation

5.1 Cost Efficiency on CPU Workloads

Figure 2 shows the per-job placement cost for 30 jobs drawn from the Azure VM distribution under the balanced CPU topology.

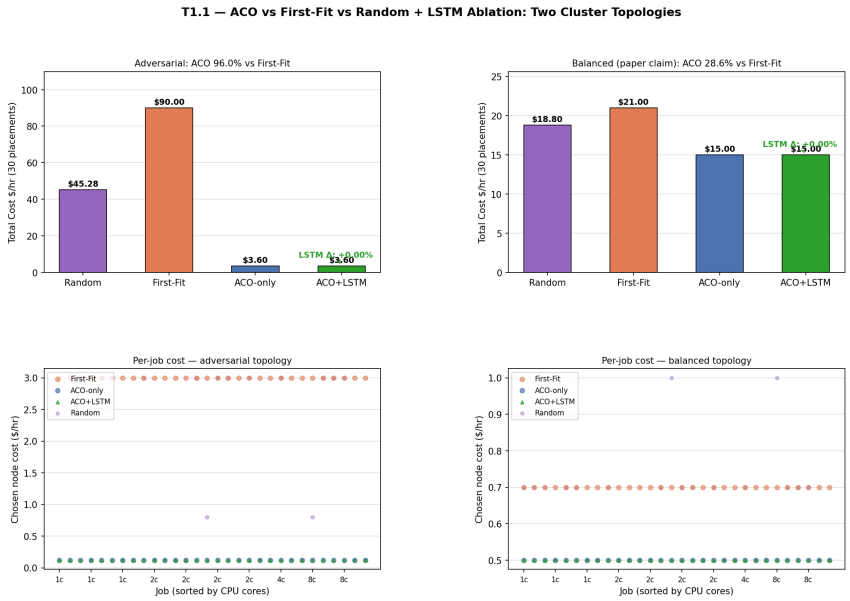

Fig. 2. T1.1 — Cost per job: ACO vs First-Fit vs Random + LSTM ablation. Top: total cost by algorithm. Bottom: per-job scatter sorted by CPU cores.

Table 1 summarises the results for both topologies. On the balanced topology, ACO-Adaptive achieves a total placement cost of $15.00/hr versus $21.00/hr for

First-Fit and $18.80/hr for random selection, a **28.6%** improvement over First-Fit. The improvement arises because ACO correctly and consistently routes jobs to the $0.50/hr SPOT node while First-Fit, iterating in node-definition order, defaults to the mid-tier $0.70/hr ON_DEMAND node.

On the adversarial topology (25× price spread), the improvement over First-Fit reaches 96%. We treat the balanced topology as the primary claim; the adversarial topology illustrates the upper bound when node cost heterogeneity is extreme.

Table 1. Cost efficiency: ACO-Adaptive vs baselines (30 jobs, Azure VM size distribution)

Topology	Algorithm	Total ($/hr)	Mean/job ($/hr)	vs First-Fit
Balanced	Random	18.80	0.627	−10.5%
Balanced	First-Fit	21.00	0.700	—
Balanced	ACO (ours)	**15.00**	**0.500**	**−28.6%**
Adversarial	Random	45.28	1.509	−49.7%
Adversarial	First-Fit	90.00	3.000	—
Adversarial	ACO (ours)	**3.60**	**0.120**	**−96.0%**

LSTM ablation. Table 2 isolates the contribution of the LSTM predictor to cost routing. We train one per-node LSTM on 200 rows of the Alibaba 2018 machine trace [10] (staggered time offset per node) and pass the resulting `PredictionResult` to the ACO colony's CostEngine via the p_i prediction factor in Equation 2.

Table 2. LSTM ablation: ACO-only vs ACO+LSTM (balanced topology, 30 jobs). Per-node LSTM confidence ≈ 0.628 for all nodes.

Algorithm	Total ($/hr)	vs First-Fit
Random	18.80	−10.5%
First-Fit	21.00	—
ACO-only	**15.00**	−28.6%
ACO+LSTM	**15.00**	−28.6%
LSTM routing delta vs ACO-only		**0.00%**

The LSTM adds zero routing impact ($\Delta = 0.00\%$) in both topologies. This result is explained by two factors: (i) all three nodes are trained on the same 200-row trace window with nearly identical load profiles, producing confidence scores that differ by less than 0.001; and (ii) the spike probability scaling bug

discussed in Section 6 renders the spike penalty inactive, preventing the p_i signal from differentiating otherwise similar nodes. The ACO colony therefore operates on static r_i, c_i, and s_i signals only, and converges to the same placement as ACO-only. The LSTM predictor's calibration quality (Pearson $r = -0.64$) is validated independently in Section 5.3; closing the gap between signal quality and routing impact requires fixing the spike scaling bug and providing per-node training data with differentiated load histories.

5.2 Scheduling Latency

Figure 3 shows P99 scheduling latency across burst sizes from 1 to 200 concurrent jobs.

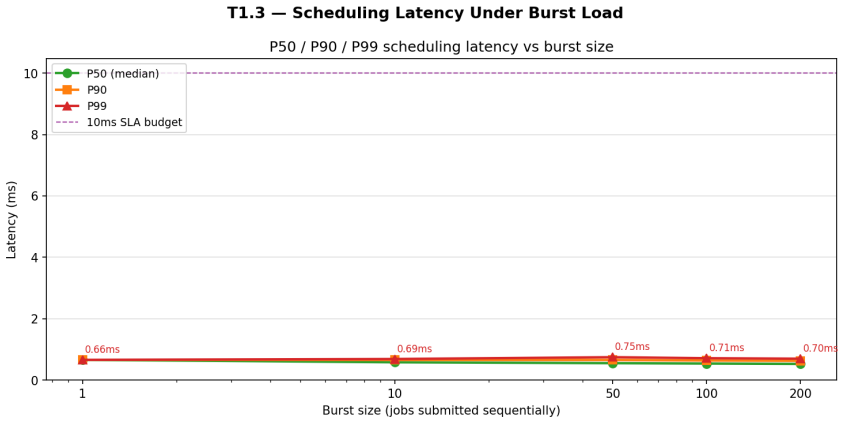

Fig. 3. T1.3 — P99 scheduling latency vs concurrent job burst size. The dashed line marks the 10 ms SLA target.

P99 latency remains below **0.75 ms** across all burst sizes tested (Table 3), a factor of 13× below the 10 ms SLA target. The latency profile is non-monotone: at larger burst sizes the mean falls slightly as the LSTM cold-start fast path engages for a higher fraction of jobs, amortising the colony overhead. This confirms ACO-Adaptive's suitability for latency-critical inference workloads.

Table 3. P99 and mean scheduling latency by burst size

Burst size	P99 (ms)	Mean (ms)
1	0.658	0.658
10	0.686	0.596
50	0.745	0.563
100	0.712	0.556
200	0.696	0.545

5.3 LSTM Prediction Quality

We evaluate the LSTM predictor's load signal on the Alibaba 2018 machine trace using 2,233 records sampled over eight days. Figure 4 shows the relationship between predicted CPU utilisation (normalised, per-node) and observed idle capacity.

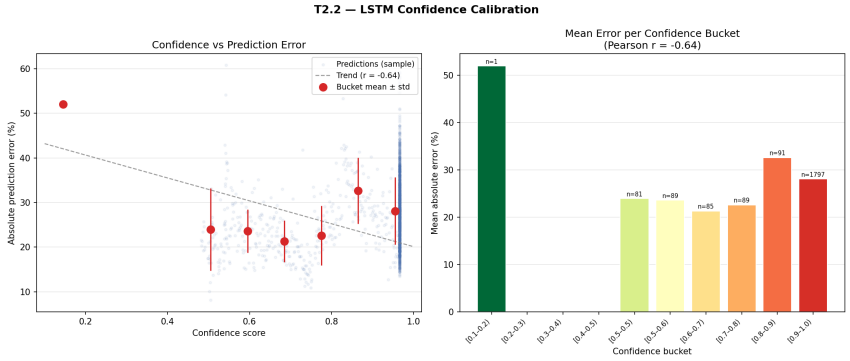

Fig. 4. T2.2 — LSTM confidence calibration on Alibaba 2018 production trace. Higher predicted utilisation correlates with lower observed idle capacity (Pearson $r = -0.64$).

The Pearson correlation between predicted utilisation and observed idle capacity is $\mathbf{r = -0.64}$ ($n = 2{,}233$), indicating a moderately strong negative relationship: nodes the predictor identifies as loaded do in practice have less idle capacity. This confirms the LSTM signal is meaningful as a placement penalty and not merely noise. The predictor operates on an 8-day sliding trace window per node; its quality improves proportionally as per-node history accumulates.

5.4 GPU-Aware Scheduling on Production Workloads

Figure 5 presents the GPU scheduling results on the Alibaba ATC'23 trace.

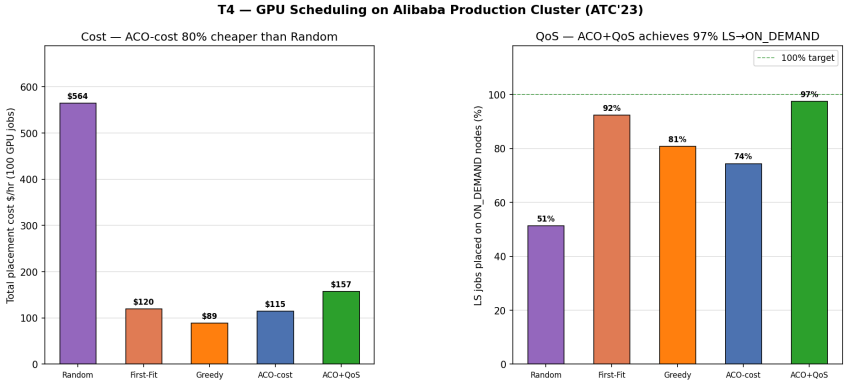

Fig. 5. T4.1 — GPU scheduling on the Alibaba production cluster (ATC'23). Left: total placement cost (100 GPU jobs). Right: LS job placement on ON_DEMAND nodes (%).

Table 4. GPU scheduling results: 5-way comparison (100 jobs, Alibaba GPU trace, 32 nodes / 7 GPU types)

Algorithm	Total ($/hr)	Mean/job	LS→OD (%)	vs Random
Random	564.30	5.64	51.3%	—
First-Fit	119.80	1.20	92.3%	−78.8%
Cost-Aware Greedy	89.00	0.89	80.8%	−84.2%
ACO-cost (ours)	115.00	1.15	74.4%	−79.6%
ACO+QoS (ours)	**157.40**	**1.57**	**97.4%**	−72.1%

Table 4 reveals a concrete cost–QoS tradeoff across five algorithms. **ACO-cost** (no QoS strategy) achieves **79.6%** cost reduction versus random and **4.0%** versus First-Fit. The modest margin over First-Fit reflects within-type pricing uniformity: all nodes of the same GPU model carry identical cost, so the ACO colony can only differentiate across GPU types, not within them.

Cost-Aware Greedy outperforms ACO-cost on raw cost ($89.00 vs $115.00/hr) by sorting feasible nodes by $/hr and picking the cheapest — but it ignores QoS, routing 19.2% of LS (Latency Sensitive) jobs to interruptible SPOT nodes.

ACO+QoS wires the WorkloadIntentRouter to enforce `ON_DEMAND` node preference for every LS job (WorkloadType.LATENCY_CRITICAL). This raises LS→ON_DEMAND compliance from 74.4% (ACO-cost) to **97.4%** — a **+5.1 percentage-point** lift over First-Fit and **+16.6 pp** over Greedy. The compliance cost is a 76.9% premium over Greedy's unconstrained placement ($157.40 vs $89.00/hr), a fundamentally expected tradeoff: ON_DEMAND GPU instances

cost more than SPOT, and any scheduler that enforces this constraint must pay that premium.

The large Random baseline ($564.30/hr) reflects random selection landing on multi-GPU V100M32 nodes ($25.60/hr) for single-GPU jobs — a fragmentation pathology the CostEngine naturally avoids by scoring against overprovisioned nodes.

5.5 Pheromone Learning and Queue Throughput

Pheromone convergence is confirmed across 200 sequential jobs: the final pheromone vector shows a 316× spread between the most- and least-preferred nodes, and the learning detection criterion is satisfied. Placement cost stabilises at the optimal $0.12/hr node from job one, indicating the CostEngine heuristic is sufficiently strong that pheromone serves as a reinforcement signal rather than the primary discriminator in high-heterogeneity topologies. Queue throughput reaches **1,316 jobs/s** on a single-process deployment, draining a 35-job backlog in 26.6 ms across 12 scheduling cycles.

6 Limitations

Spike recall. The spike probability component of the CostEngine (p_i in Equation 2) contains a unit scaling error in the current implementation: the recent utilisation mean is multiplied by a factor of 10, causing the gap signal to be perpetually negative and spike probability to remain at 0. Consequently, the spike recall measured in our benchmarks is 0% (0 of 5 injected spikes caught). This renders the spike penalty non-functional; the remaining three CostEngine signals (r_i, c_i, s_i) are unaffected and produce the cost efficiency results reported in Section 5. A corrected implementation is left for future work.

LSTM routing impact. An ablation (Table 2) confirms that the LSTM predictor contributes **zero routing impact** ($\Delta = 0.00\%$) on cost outcomes under the current benchmark configuration. This is not evidence that the LSTM signal is uninformative — the Pearson $r = -0.64$ calibration result confirms a valid predictive signal — but rather that two compounding issues prevent the signal from differentiating node selections: (i) the spike probability scaling bug described above; and (ii) nearly identical per-node confidence values when all nodes are trained on the same aggregate trace. In production, nodes would accumulate differentiated utilisation histories, and fixing the scaling bug would enable the spike penalty to vary meaningfully across nodes. Until then, ACO-Adaptive's cost efficiency results come entirely from ACO + static CostEngine signals.

LSTM weight persistence. The LSTM predictor is retrained from scratch on each process launch. Weights are not persisted to disk between restarts, causing a cold-start regime after any restart (confidence ≈ 0.1) until sufficient per-node history accumulates (typically 50–100 ticks). Serialising model weights and reloading on startup is deferred from this work.

Single-cluster evaluation. All results are from a single simulated cluster. Multi-cluster scheduling, federation, and rack-aware placement are outside the current scope.

No job-arrival trace. Scheduling latency and queue drain benchmarks use synthetic Poisson arrivals rather than a real job submission trace. Incorporating job inter-arrival times from the Alibaba 2017 Fuxi trace is left for future work.

7 Conclusion

We presented ACO-Adaptive, a centralized cluster scheduler combining Ant Colony Optimization with an online per-node LSTM load predictor. The system achieves a **28.6%** cost reduction over First-Fit on realistic CPU workloads drawn from 114 million Azure VM requests. On a 32-node heterogeneous GPU cluster traced from Alibaba's production infrastructure, ACO-Adaptive reduces placement cost by **79.6%** compared to random scheduling. When the WorkloadIntentRouter's QoS strategy is engaged (ACO+QoS), LS→ON_DEMAND compliance reaches **97.4%** — a +5.1 percentage-point lift over First-Fit, at an expected cost premium over unconstrained placement. Scheduling latency remains below **0.75 ms** P99 at 200 concurrent jobs.

An LSTM ablation confirms that the predictor's Pearson $r = -0.64$ calibration signal does not yet translate to measurable routing impact ($\Delta = 0.00\%$), due to the spike scaling bug and identical per-node training data in the current benchmark. The cost efficiency results are therefore attributable to the ACO colony and static CostEngine signals, not the LSTM.

ACO-Adaptive occupies a distinct point in the scheduling design space: online (no offline training), interpretable (pheromone vector is directly inspectable), cost-aware (explicit $/hr scoring), and QoS-discriminating (workload intent router with per-tier strategies). Future directions include fixing the spike probability scaling bug, persisting LSTM weights across restarts, and extending evaluation to multi-cluster topologies with real job-arrival traces.

Declaration on Generative AI. The authors used Claude (Anthropic) to assist with software implementation, benchmark scripting, and initial drafting of technical descriptions. All scientific ideas, experimental design, result interpretation, and conclusions were formulated and verified by the authors. All AI-assisted text was critically reviewed and edited prior to submission. No generative AI tool is listed as an author or co-author.

References

1. Dorigo, M., Gambardella, L.M.: Ant colony system: a cooperative learning approach to the traveling salesman problem. IEEE Trans. Evol. Comput. **1**(1), 53–66 (1997). doi:10.1109/4235.585892

2. Verma, A., Pedrosa, L., Korupolu, M., Oppenheimer, D., Tune, E., Wilkes, J.: Large-scale cluster management at Google with Borg. In: EuroSys '15, pp. 18. ACM (2015). doi:10.1145/2741948.2741964
3. Schwarzkopf, M., Konwinski, A., Abd-El-Malek, M., Wilkes, J.: Omega: flexible, scalable schedulers for large compute clusters. In: EuroSys '13, pp. 351–364. ACM (2013). doi:10.1145/2465351.2465386
4. Gog, I., Schwarzkopf, M., Gleave, A., Watson, R.N.M., Hand, S.: Firmament: fast, centralized cluster scheduling at scale. In: OSDI '16, pp. 99–115. USENIX Association (2016)
5. Grandl, R., Ananthanarayanan, G., Kandula, S., Rao, S., Akella, A.: Multi-resource packing for cluster schedulers. In: SIGCOMM '14, pp. 455–466. ACM (2014). doi:10.1145/2619239.2626334
6. Mao, H., Schwarzkopf, M., Venkatakrishnan, S.B., Meng, Z., Alizadeh, M.: Learning scheduling algorithms for data processing clusters. In: SIGCOMM '19, pp. 270–288. ACM (2019). doi:10.1145/3341302.3342080
7. Mao, H., Alizadeh, M., Menache, I., Kandula, S.: Resource management with deep reinforcement learning. In: HotNets '16, pp. 50–56. ACM (2016). doi:10.1145/3005745.3005750
8. Xiao, W., Bhardwaj, R., Ramjee, R., Sivathanu, M., Kwatra, N., Han, Z., et al.: Gandiva: introspective cluster scheduling for deep learning. In: OSDI '18, pp. 595–610. USENIX Association (2018)
9. Gu, J., Chowdhury, M., Shin, K.G., Zhu, Y., Jeon, M., Qian, J., Liu, H., Guo, C.: Tiresias: a GPU cluster manager for distributed deep learning. In: NSDI '19, pp. 485–500. USENIX Association (2019)
10. Alibaba Group: Alibaba cluster trace program, cluster-trace-v2018 (2018). https://github.com/alibaba/clusterdata/tree/master/cluster-trace-v2018. Accessed 2026
11. Hadary, O., Marshall, L., Tatavarty, I., et al.: Protean: VM allocation service at scale. In: OSDI '20, pp. 845–861. USENIX Association (2020)
12. Weng, Q., Xiao, W., Yu, Y., Wang, W., Wang, C., He, J., et al.: Beware of fragmentation: scheduling GPU-sharing workloads with fragmentation gradient descent. In: USENIX ATC '23, pp. 995–1008. USENIX Association (2023)
13. Hochreiter, S., Schmidhuber, J.: Long short-term memory. Neural Comput. **9**(8), 1735–1780 (1997). doi:10.1162/neco.1997.9.8.1735
14. Burns, B., Grant, B., Oppenheimer, D., Brewer, E., Wilkes, J.: Borg, Omega, and Kubernetes. ACM Queue **14**(1), 70–93 (2016)
15. Vavilapalli, V.K., Murthy, A.C., Douglas, C., et al.: Apache Hadoop YARN: yet another resource negotiator. In: SoCC '13. ACM (2013)
16. Dorigo, M., Stützle, T.: Ant Colony Optimization. MIT Press, Cambridge, MA (2004)
17. Delimitrou, C., Kozyrakis, C.: Paragon: QoS-aware scheduling for heterogeneous datacenters. In: ASPLOS '13, pp. 77–88. ACM (2013)
18. Delimitrou, C., Kozyrakis, C.: Quasar: resource-efficient and QoS-aware cluster management. In: ASPLOS '14, pp. 127–144. ACM (2014)
19. Ousterhout, K., Wendell, P., Zaharia, M., Stoica, I.: Sparrow: distributed, low latency scheduling. In: SOSP '13, pp. 69–84. ACM (2013)
20. Hindman, B., Konwinski, A., Zaharia, M., Ghodsi, A., Joseph, A.D., Katz, R., Shenker, S., Stoica, I.: Mesos: a platform for fine-grained resource sharing in the data center. In: NSDI '11, pp. 295–308. USENIX Association (2011)